From Separation Logic to Rust

Ownership as a type system — and what it costs

Cristina David

Lecture 13

COMSM0067 — Advanced Topics in Programming Languages
University of Bristol

Five lectures of machinery

- Hoare logic: specify with triples, prove with six rules.
- Invariants and wp: automate the bookkeeping, hand the logic to a solver.
- Separation logic: $*$ and the frame rule, to reason *locally* about aliased mutable memory.
- Symbolic execution: the same machinery forwards, hunting bugs.

Five lectures of machinery

- Hoare logic: specify with triples, prove with six rules.
- Invariants and wp: automate the bookkeeping, hand the logic to a solver.
- Separation logic: $*$ and the frame rule, to reason *locally* about aliased mutable memory.
- Symbolic execution: the same machinery forwards, hunting bugs.

All of it costs effort: annotations, invariants, proofs. Someone must write them.

What if the language did it for you?

The question for today

We built a discipline for reasoning about
aliased, mutable memory.

What if a language baked that discipline into its type system —
so the checking happened at compile time,
with no proofs to write?

The question for today

We built a discipline for reasoning about
aliased, mutable memory.

What if a language baked that discipline into its type system —
so the checking happened at compile time,
with no proofs to write?

That language exists, and millions of people use it.

Rust in three ideas

No Rust assumed

The rest of this lecture compares Rust's type system with separation logic. So first, the three ideas you need — and nothing more.

1. Ownership — who is responsible for a value.
2. Borrowing — lending access without giving it away.
3. The rule — how many borrows may coexist.

No Rust assumed

The rest of this lecture compares Rust's type system with separation logic. So first, the three ideas you need — and nothing more.

1. Ownership — who is responsible for a value.
2. Borrowing — lending access without giving it away.
3. The rule — how many borrows may coexist.

If you already write Rust, the next three slides will be familiar. Everything after them is not.

(1) Ownership

Every value has exactly one owner. Assignment moves ownership — and the old name stops working.

```
let a = String::from("hi");
let b = a;       // ownership moves to b
print(a);        // compile error:  a no longer owns it
```

(1) Ownership

Every value has exactly one owner. Assignment moves ownership — and the old name stops working.

```
let a = String::from("hi");
let b = a;       // ownership moves to b
print(a);        // compile error:  a no longer owns it
```

Why so strict? If both `a` and `b` owned the value, both would free it when they went out of scope — a double free. One owner, one release.

(Small integers are *copied* rather than moved — a special case we can ignore today.)

(2) Borrowing

Moving is drastic. Usually you just want to *use* a value for a while — so you take a borrow, written `&`.

```
let a = String::from("hi");
let r = &a;     // borrow:  r reads a; a still owns it
print(a);      // fine -- a was never given away
```

(2) Borrowing

Moving is drastic. Usually you just want to *use* a value for a while — so you take a borrow, written `&`.

```
let a = String::from("hi");
let r = &a;     // borrow:  r reads a; a still owns it
print(a);      // fine -- a was never given away
```

	written	gives you
shared borrow	`&T`	read access; many at once
unique borrow	`&mut T`	read *and* write; only one

A borrow is temporary: when it ends, full access returns to the owner.

(3) The rule

At any moment, for any value:

either one `&mut`, or any number of `&`.

Never both.

(3) The rule

At any moment, for any value:

either one `&mut`, or any number of `&`.

Never both.

Reading is safe to share — nothing changes underneath you. Writing is not.

(3) The rule

At any moment, for any value:

either one `&mut`, or any number of `&`.

Never both.

Reading is safe to share — nothing changes underneath you. Writing is not.

So the rule forbids exactly one thing: aliasing together with mutation.

Which should sound familiar — it is what broke the assignment axiom in Lecture 10.

Ownership is the points-to assertion

The correspondence

Separation logic	Rust
$x \mapsto v$ — I own this cell	`T` — owned value (move semantics)
exclusive write permission	`&mut T` — unique borrow
read-only share	`&T` — shared borrow
$P * Q$ — disjoint footprints	a `&mut`, disjoint from all others
the frame rule	a function signature

The correspondence

Separation logic	Rust
$x \mapsto v$ — I own this cell	`T` — owned value (move semantics)
exclusive write permission	`&mut T` — unique borrow
read-only share	`&T` — shared borrow
$P * Q$ — disjoint footprints	a `&mut`, disjoint from all others
the frame rule	a function signature

The borrow checker is a decision procedure for a fragment of separation logic.

(Row 4 simplifies: $*$ is *exclusive*, so it models `&mut` exactly. Many readers at once needs fractional permissions — not covered here.)

Ownership: the core rule

Rust's central discipline, in one line:

> Either one mutable reference, or any number of shared ones. Never both.

Ownership: the core rule

Rust's central discipline, in one line:

Either one mutable reference, or any number of shared ones. Never both.

Read it as separation logic. A mutable borrow `&mut x` is a claim to exclusive ownership of x's cell — so nothing else may hold it. That is precisely the disjointness $*$ asserts.

Ownership: the core rule

Rust's central discipline, in one line:

> Either one mutable reference, or any number of shared ones. Never both.

Read it as separation logic. A mutable borrow `&mut x` is a claim to exclusive ownership of x's cell — so nothing else may hold it. That is precisely the disjointness $*$ asserts.

Why this rule and not another?

Because aliasing + mutation is exactly what broke the assignment axiom in Lecture 10. Forbid their combination and substitution-style reasoning works again — which is what the compiler exploits.

Borrowing, checked

```
let mut s = String::from("hi");
let a = &s;      // shared
let b = &s;      // shared -- fine, both only read
```

Borrowing, checked

```
let mut s = String::from("hi");
let a = &s;      // shared
let b = &s;      // shared -- fine, both only read


let mut s = String::from("hi");
let a = &mut s;  // unique
let b = &s;      // rejected -- already borrowed uniquely
```

Borrowing, checked

```
let mut s = String::from("hi");
let a = &s;       // shared
let b = &s;       // shared -- fine, both only read


let mut s = String::from("hi");
let a = &mut s;   // unique
let b = &s;       // rejected -- already borrowed uniquely
```

In separation logic you would have had to *prove* the footprints disjoint. Here the type system rejects the program that would have made them overlap — no proof obligation, because the bad state is unrepresentable.

A signature is a frame rule

The frame rule, as a function signature

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}\ \text{Frame}$$

Now read a Rust signature:

```
fn f(x:  &mut T)
```

The frame rule, as a function signature

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}\ \text{Frame}$$

Now read a Rust signature:

```
fn f(x:  &mut T)
```

- `f` may touch what it was given — that is P.
- Everything else is R, and `f` cannot name it, so it cannot touch it.
- The caller keeps R unchanged, for free.

The frame rule, as a function signature

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}\ \text{Frame}$$

Now read a Rust signature:

```
fn f(x:  &mut T)
```

- `f` may touch what it was given — that is P.
- Everything else is R, and `f` cannot name it, so it cannot touch it.
- The caller keeps R unchanged, for free.

The type signature *is* the frame.

Who supplies the footprint?

In Lectures 10–11 the footprint was yours to write. Rust collects the same information in the type:

```
fn f(x:  &mut T)
```

Who supplies the footprint?

In Lectures 10–11 the footprint was yours to write. Rust collects the same information in the type:

```
fn f(x:  &mut T)
```

`f` can reach none of the caller's data except through `x`.

Separation logic	**Rust**
footprint written as a precondition	footprint written as a *type*
frame rule applied by you	framing applied by the compiler
discharged by a proof	discharged by type-checking

Who supplies the footprint?

In Lectures 10–11 the footprint was yours to write. Rust collects the same information in the type:

```
fn f(x:  &mut T)
```

`f` can reach none of the caller's data except through `x`.

Separation logic	**Rust**
footprint written as a precondition	footprint written as a *type*
frame rule applied by you	framing applied by the compiler
discharged by a proof	discharged by type-checking

("The caller's data" is the honest limit: `f` may still allocate, do I/O, or touch global state — ambient, and named in no signature.)

The price

Nothing comes for free

The borrow checker is automatic, fast, and needs no proofs.

So what did we give up?

Nothing comes for free

The borrow checker is automatic, fast, and needs no proofs.

So what did we give up?

Completeness.

Nothing comes for free

The borrow checker is automatic, fast, and needs no proofs.

So what did we give up?

Completeness.

There are programs that are perfectly safe and that the borrow checker rejects.

Safe, but rejected

```
let mut v = vec![1,2,3];    // a growable array
let a = &mut v[0];
let b = &mut v[1];      // rejected
```

Two *different* elements — genuinely disjoint, exactly as the two halves were a moment ago. Nothing can go wrong. But this time it is rejected.

Safe, but rejected

```
let mut v = vec![1,2,3];    // a growable array
let a = &mut v[0];
let b = &mut v[1];      // rejected
```

Two *different* elements — genuinely disjoint, exactly as the two halves were a moment ago. Nothing can go wrong. But this time it is rejected.

But the borrow checker reasons about v, not about indices. To accept this it would have to prove $0 \neq 1$ — and in general, to prove arbitrary arithmetic facts about index expressions.

That is a theorem prover, not a type checker.

This is Lecture 8, returning

Recall: Hoare logic is only relatively complete (Cook 1978). Completeness required an oracle for the entailments in Conseq — and by Gödel, no effective procedure is that oracle.

This is Lecture 8, returning

Recall: Hoare logic is only relatively complete (Cook 1978). Completeness required an oracle for the entailments in Conseq — and by Gödel, no effective procedure is that oracle.

The borrow checker is an effective procedure. It always terminates, it needs no annotations, it never asks you for an invariant.

So it cannot be complete. The rejections are not bugs — they are the theorem.

This is Lecture 8, returning

Recall: Hoare logic is only relatively complete (Cook 1978). Completeness required an oracle for the entailments in Conseq — and by Gödel, no effective procedure is that oracle.

The borrow checker is an effective procedure. It always terminates, it needs no annotations, it never asks you for an invariant.

So it cannot be complete. The rejections are not bugs — they are the theorem.

Every automatic checker sits somewhere on this trade-off. Rust chose: decidable, fast, sound, deliberately incomplete.

Sound in which direction?

	approximates	errs by
Symbolic execution (L12)	under	missing real bugs
Borrow checker	over	rejecting safe programs

Both are sound — but for opposite tasks.

Sound in which direction?

	approximates	errs by
Symbolic execution (L12)	under	missing real bugs
Borrow checker	over	rejecting safe programs

Both are sound — but for opposite tasks.

- Symbolic execution: every bug it reports is real. Says nothing about what it did not explore.
- Borrow checker: every program it accepts is safe. Says nothing kind about some programs that are also safe.

"Sound" is always *sound with respect to a claim*. Ask which claim.

The escape hatch

unsafe

`unsafe` does not switch the borrow checker off. It permits exactly five operations you otherwise cannot write:

- dereference a raw pointer;
- call an `unsafe` function;
- read or write a `static mut`;
- implement an `unsafe` trait;
- read a field of a `union`.

unsafe

`unsafe` does not switch the borrow checker off. It permits exactly five operations you otherwise cannot write:

- dereference a raw pointer;
- call an `unsafe` function;
- read or write a `static mut`;
- implement an `unsafe` trait;
- read a field of a `union`.

Everything else is checked exactly as usual — including borrow checking of ordinary references, inside the `unsafe` block as much as outside it.

`unsafe` does not switch the borrow checker off. It permits exactly five operations you otherwise cannot write:

- dereference a raw pointer;
- call an `unsafe` function;
- read or write a `static mut`;
- implement an `unsafe` trait;
- read a field of a `union`.

Everything else is checked exactly as usual — including borrow checking of ordinary references, inside the `unsafe` block as much as outside it.

What `unsafe` means

For *those five*, the proof obligation moves from the compiler to you. You are asserting a theorem the checker cannot verify.

First: what is undefined behaviour?

The language definition says what a program means *provided* it obeys certain rules — no use-after-free, no data races, no aliasing violations. Break one, and the standard assigns your program no meaning at all.

First: what is undefined behaviour?

The language definition says what a program means *provided* it obeys certain rules — no use-after-free, no data races, no aliasing violations. Break one, and the standard assigns your program no meaning at all.

That is worse than an error, because nothing is obliged to tell you:

- it may do exactly what you expected — today, on this compiler;
- it may break when you enable optimisations, or upgrade the compiler;
- the compiler may delete your code, having assumed the rule held.

First: what is undefined behaviour?

The language definition says what a program means *provided* it obeys certain rules — no use-after-free, no data races, no aliasing violations. Break one, and the standard assigns your program no meaning at all.

That is worse than an error, because nothing is obliged to tell you:

- it may do exactly what you expected — today, on this compiler;
- it may break when you enable optimisations, or upgrade the compiler;
- the compiler may delete your code, having assumed the rule held.

UB is not "a crash". It is the loss of any guarantee whatsoever — including about the parts of the program that looked unrelated.

First: what is undefined behaviour?

The language definition says what a program means *provided* it obeys certain rules — no use-after-free, no data races, no aliasing violations. Break one, and the standard assigns your program no meaning at all.

That is worse than an error, because nothing is obliged to tell you:

- it may do exactly what you expected — today, on this compiler;
- it may break when you enable optimisations, or upgrade the compiler;
- the compiler may delete your code, having assumed the rule held.

UB is not "a crash". It is the loss of any guarantee whatsoever — including about the parts of the program that looked unrelated.

Safe Rust cannot cause it. `unsafe` can — which is what the rest of this section is about.

A complete example

A raw pointer is Rust's untracked pointer — the C kind. Creating one is safe; using one needs `unsafe`, because the compiler tracks nothing about it.

```
let mut x = 42;
let p = &mut x as *mut i32;    // raw pointer to x
let r1 = unsafe { &mut *p };
*r1 = 1;
let r2 = unsafe { &mut *p };
*r2 = 2;
```

A complete example

A raw pointer is Rust's untracked pointer — the C kind. Creating one is safe; using one needs `unsafe`, because the compiler tracks nothing about it.

```
let mut x = 42;
let p = &mut x as *mut i32;     // raw pointer to x
let r1 = unsafe { &mut *p };
*r1 = 1;
let r2 = unsafe { &mut *p };
*r2 = 2;
```

Two `&mut` to the same `x`, both made from the same raw pointer.

This program is fine. No undefined behaviour.

The same lines, reordered

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };    // both made first
*r2 = 2;
*r1 = 1;                        // undefined behaviour
```

The same lines, reordered

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };    // both made first
*r2 = 2;
*r1 = 1;                        // undefined behaviour
```

Same six lines. Only the order differs — and now the program is broken.

The same lines, reordered

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };    // both made first
*r2 = 2;
*r1 = 1;                        // undefined behaviour
```

Same six lines. Only the order differs — and now the program is broken.

Nothing in the type checker distinguishes these. So what does?

Stacked Borrows

Stacked Borrows (Jung et al., POPL 2020) gives every memory location a stack of tags — one per pointer or reference entitled to touch it. Two rules:

- Making a reference from a pointer p: find p's tag, pop everything above it, then push the new tag on top.
- Using a reference with tag t: t must be somewhere in the stack. If it is not — undefined behaviour.

Stacked Borrows

Stacked Borrows (Jung et al., POPL 2020) gives every memory location a stack of tags — one per pointer or reference entitled to touch it. Two rules:

- Making a reference from a pointer p: find p's tag, pop everything above it, then push the new tag on top.
- Using a reference with tag t: t must be somewhere in the stack. If it is not — undefined behaviour.

So a new reference does not *coexist* with the old one. It evicts it.

Stacked Borrows

Stacked Borrows (Jung et al., POPL 2020) gives every memory location a stack of tags — one per pointer or reference entitled to touch it. Two rules:

- Making a reference from a pointer p: find p's tag, pop everything above it, then push the new tag on top.
- Using a reference with tag t: t must be somewhere in the stack. If it is not — undefined behaviour.

So a new reference does not *coexist* with the old one. It evicts it.

(Simplified: the real model distinguishes unique from shared tags, and reads from writes.)

Tracing the good one

line	stack afterwards	
`let p = &mut x as *mut i32;`	$[p]$	
`let r1 = unsafe { &mut *p };`	$[p, r_1]$	
`*r1 = 1;`	$[p, r_1]$	— r_1 is in the stack ✓
`let r2 = unsafe { &mut *p };`	$[p, r_2]$	— r_1 popped
`*r2 = 2;`	$[p, r_2]$	— r_2 is in the stack ✓

Tracing the good one

line	stack afterwards	
`let p = &mut x as *mut i32;`	$[p]$	
`let r1 = unsafe { &mut *p };`	$[p, r_1]$	
`*r1 = 1;`	$[p, r_1]$	— r_1 is in the stack ✓
`let r2 = unsafe { &mut *p };`	$[p, r_2]$	— r_1 popped
`*r2 = 2;`	$[p, r_2]$	— r_2 is in the stack ✓

Creating r_2 *did* evict r_1 — but we had already finished with r_1.

Sequential use. No violation.

Tracing the bad one

line	stack afterwards
`let p = &mut x as *mut i32;`	$[p]$
`let r1 = unsafe { &mut *p };`	$[p,\ r_1]$
`let r2 = unsafe { &mut *p };`	$[p,\ r_2]$ — r_1 popped
`*r2 = 2;`	$[p,\ r_2]$ — fine ✓
`*r1 = 1;`	r_1 is not in the stack $\Rightarrow$ UB

Tracing the bad one

line	stack afterwards
`let p = &mut x as *mut i32;`	$[p]$
`let r1 = unsafe { &mut *p };`	$[p,\ r_1]$
`let r2 = unsafe { &mut *p };`	$[p,\ r_2]$ — r_1 popped
`*r2 = 2;`	$[p,\ r_2]$ — fine ✓
`*r1 = 1;`	r_1 is not in the stack ⇒ UB

The error is using r_1 after it was evicted — not creating r_2, and not the write through r_2.

Tracing the bad one

line	stack afterwards
`let p = &mut x as *mut i32;`	$[p]$
`let r1 = unsafe { &mut *p };`	$[p,\ r_1]$
`let r2 = unsafe { &mut *p };`	$[p,\ r_2]$ — r_1 popped
`*r2 = 2;`	$[p,\ r_2]$ — fine ✓
`*r1 = 1;`	r_1 is not in the stack ⇒ UB

The error is using r_1 after it was evicted — not creating r_2, and not the write through r_2.

Informally: "never two live `&mut`". Precisely: r_1 stopped being *live* the moment r_2 was made.

Miri

The rules are operational — so they can be run. Miri interprets your program while maintaining the stacks, and reports the violation:

```
error: Undefined Behavior: attempting a write access
    using <t1> but that tag does not exist in the
    borrow stack for this location
```

Miri

The rules are operational — so they can be run. Miri interprets your program while maintaining the stacks, and reports the violation:

```
error: Undefined Behavior: attempting a write access
    using <t1> but that tag does not exist in the
    borrow stack for this location
```

It points at `*r1 = 1` — exactly where the trace said.

Miri

The rules are operational — so they can be run. Miri interprets your program while maintaining the stacks, and reports the violation:

```
error: Undefined Behavior: attempting a write access
    using <t1> but that tag does not exist in the
    borrow stack for this location
```

It points at `*r1 = 1` — exactly where the trace said.

Why you need it

The bad program very often *prints the right answer*. Nothing detects it at compile time, and testing the output will not either. Miri checks the rule, not the outcome.

And what does Rust actually *prove*?

Rust's safety theorem is progress and preservation — the same shape you proved in the first half of this unit.

And what does Rust actually *prove*?

Rust's safety theorem is progress and preservation — the same shape you proved in the first half of this unit.

It guarantees: no use-after-free, no data races, no dangling pointers, no double-free.

And what does Rust actually *prove*?

Rust's safety theorem is progress and preservation — the same shape you proved in the first half of this unit.

It guarantees: no use-after-free, no data races, no dangling pointers, no double-free.

What it does *not* prove

That your program computes the right answer. A sorting function that returns its input unchanged is perfectly memory-safe.

Which is exactly where we came in, in Lecture 8.

Closing

The whole arc

Lecture	The idea
8	Say what a program *should* do; prove it. Manual.
9	Automate the bookkeeping; a human still supplies invariants.
10	The heap breaks substitution. $*$ and framing restore locality.
11	Recursive predicates; a real proof about a real algorithm.
12	Run it forwards: under-approximate, and find real bugs.
13	Put the discipline in the *type system*; pay in completeness.

The whole arc

Lecture	The idea
8	Say what a program *should* do; prove it. Manual.
9	Automate the bookkeeping; a human still supplies invariants.
10	The heap breaks substitution. $*$ and framing restore locality.
11	Recursive predicates; a real proof about a real algorithm.
12	Run it forwards: under-approximate, and find real bugs.
13	Put the discipline in the *type system*; pay in completeness.

One question, six answers: what does it take to reason about an assignment?

The borrow checker is
separation logic
you did not have to write.

The borrow checker is
separation logic
you did not have to write.

And the price of not writing it is that sometimes
it says *no* when the answer was *yes*.

Where this goes next

- RustBelt (POPL 2018) — a machine-checked proof, in Coq, that Rust's type system is sound, *including* the `unsafe` standard library. The trusted core, finally proved. Built on separation logic (Iris).
- Verus, Prusti, Creusot — full functional verification *for* Rust: ownership handles the memory reasoning, so the prover only has to do the arithmetic.

Where this goes next

- RustBelt (POPL 2018) — a machine-checked proof, in Coq, that Rust's type system is sound, *including* the `unsafe` standard library. The trusted core, finally proved. Built on separation logic (Iris).
- Verus, Prusti, Creusot — full functional verification *for* Rust: ownership handles the memory reasoning, so the prover only has to do the arithmetic.

Problem Sheet 13 is out: borrow-checker verdicts, SL-to-Rust translation, and writing down the proof obligation you take on with `unsafe`.

Thank you.

Questions?

Backup: dividing a permission

The ownership rule says one `&mut` at a time. So how do you ever work on two halves of an array at once — say, sort each half independently?

Backup: dividing a permission

The ownership rule says one `&mut` at a time. So how do you ever work on two halves of an array at once — say, sort each half independently?

The halves are disjoint. Nothing can go wrong. But the rule appears to forbid it.

Backup: dividing a permission

The ownership rule says one `&mut` at a time. So how do you ever work on two halves of an array at once — say, sort each half independently?

The halves are disjoint. Nothing can go wrong. But the rule appears to forbid it.

The next slides are the answer, and it is $*$.

Is the rule too strong?

Only one `&mut` to a value at a time. So consider an ordinary task: sort the two halves of an array independently — perhaps on two threads.

Is the rule too strong?

Only one `&mut` to a value at a time. So consider an ordinary task: sort the two halves of an array independently — perhaps on two threads.

That needs two mutable borrows of the same array, live at once. The rule appears to forbid it.

Is the rule too strong?

Only one `&mut` to a value at a time. So consider an ordinary task: sort the two halves of an array independently — perhaps on two threads.

That needs two mutable borrows of the same array, live at once. The rule appears to forbid it.

And yet the halves are disjoint. Nothing can go wrong.

Is the rule too strong?

Only one `&mut` to a value at a time. So consider an ordinary task: sort the two halves of an array independently — perhaps on two threads.

That needs two mutable borrows of the same array, live at once. The rule appears to forbid it.

And yet the halves are disjoint. Nothing can go wrong.

So: can ownership express "different parts of the same thing"?

Yes — by splitting the permission

```
let (l, r) = v.split_at_mut(mid);
```

One `&mut` to the whole array becomes two `&mut`s, to the left and right halves — both live, and accepted.

Yes — by splitting the permission

```
let (l, r) = v.split_at_mut(mid);
```

One `&mut` to the whole array becomes two `&mut`s, to the left and right halves — both live, and accepted.

This is $*$, at run time. Writing $\mathrm{own}(v)$ for the permission to v:

$$\mathrm{own}(v) \quad \rightsquigarrow \quad \mathrm{own}(l) * \mathrm{own}(r)$$

Yes — by splitting the permission

```
let (l, r) = v.split_at_mut(mid);
```

One `&mut` to the whole array becomes two `&mut`s, to the left and right halves — both live, and accepted.

This is $*$, at run time. Writing $\mathrm{own}(v)$ for the permission to v:

$$\mathrm{own}(v) \quad \rightsquigarrow \quad \mathrm{own}(l) * \mathrm{own}(r)$$

The rule was never "one `&mut` per *variable*". It is one per location — and $*$ is what lets you re-carve the locations.

Yes — by splitting the permission

```
let (l, r) = v.split_at_mut(mid);
```

One `&mut` to the whole array becomes two `&mut`s, to the left and right halves — both live, and accepted.

This is $*$, at run time. Writing $\mathrm{own}(v)$ for the permission to v:

$$\mathrm{own}(v) \quad \rightsquigarrow \quad \mathrm{own}(l) \ * \ \mathrm{own}(r)$$

The rule was never "one `&mut` per *variable*". It is one per location — and $*$ is what lets you re-carve the locations.

When the borrows expire you get the whole back — that reassembly is the magic wand $-\!\!*$ from Lecture 11.

So how do you write the rejected program?

Recall what the checker refused:

```
let a = &mut v[0];
let b = &mut v[1];    // rejected
```

So how do you write the rejected program?

Recall what the checker refused:

```
let a = &mut v[0];
let b = &mut v[1];    // rejected
```

You write it like this instead — and it is accepted:

```
let (l, r) = v.split_at_mut(1);
let a = &mut l[0];    // v[0]
let b = &mut r[0];    // v[1]
```

What actually changed

Not the borrow checker. It still cannot prove $0 \neq 1$, and still does no index arithmetic.

What actually changed

Not the borrow checker. It still cannot prove $0 \neq 1$, and still does no index arithmetic.

What moved is where the disjointness lives:

rejected version	**accepted version**
disjoint because $0 \neq 1$ — a fact about indices	disjoint because `l` and `r` are two separate values — a fact about types

What actually changed

Not the borrow checker. It still cannot prove $0 \neq 1$, and still does no index arithmetic.

What moved is where the disjointness lives:

rejected version	**accepted version**
disjoint because $0 \neq 1$ — a fact about indices	disjoint because `l` and `r` are two separate values — a fact about types

The same truth, restated in the vocabulary the checker speaks.

What actually changed

Not the borrow checker. It still cannot prove $0 \neq 1$, and still does no index arithmetic.

What moved is where the disjointness lives:

rejected version	**accepted version**
disjoint because $0 \neq 1$ — a fact about indices	disjoint because `l` and `r` are two separate values — a fact about types

The same truth, restated in the vocabulary the checker speaks.

But someone had to pay for the translation.